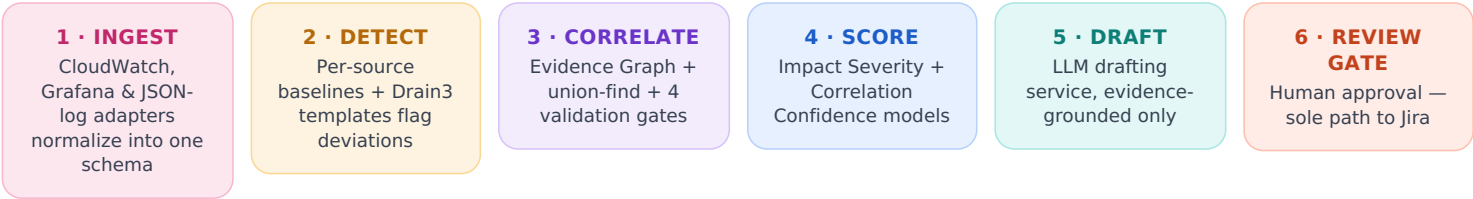


System Architecture — End-to-End & Capability Mapping

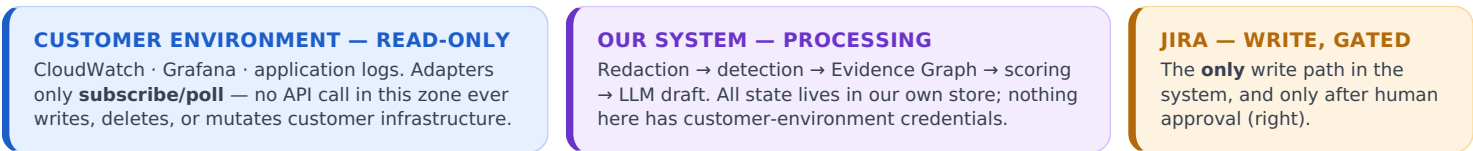
The Six Challenge Capabilities → Our Components



Redaction Boundary — Sensitive Data Never Reaches Processing



Read-Only vs. Write Boundary



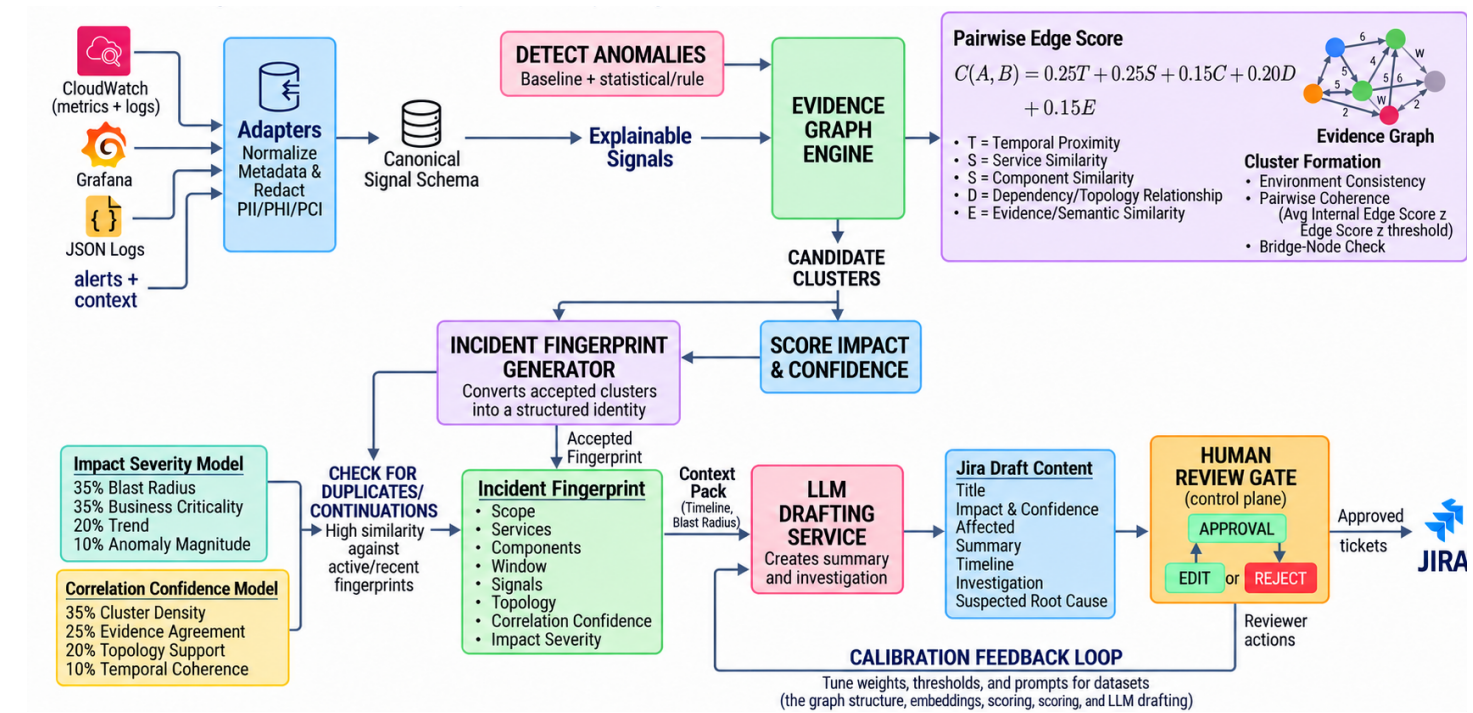
Approval Path — The Only Way to Jira



✗ LLM Draft → Jira (direct) is never a valid path in this system — every publish has a human approval record.

Failure Handling

If topology data, service metadata, the LLM, or Jira is unavailable, the system degrades — it never blocks. Missing topology/metadata → lower confidence score, not an invented value. LLM outage → evidence pack still queued for manual drafting. Jira outage → approved tickets queue and retry; nothing is silently dropped or auto-published as a workaround.



Evidence Graph — Definition, Edge Types & Validation Gates

WHAT THE GRAPH IS

Nodes = individual detected anomalies (one per signal) — each carries timestamp, environment, service, component, signal type, anomaly magnitude, and redacted evidence text. **Edges** = a weighted evidentiary relationship between two nodes, scored by $C(A,B)$ below. The graph itself never merges anything — it only *proposes* candidate structure for the gates on this page to accept or reject.

Edge Types — What Each Relationship Captures

TEMPORAL · T

Time-decay weight inside the correlation window (MVP: 5 min). Closer in time = higher score.

SERVICE / COMPONENT · S,C

Exact match, parent-child service relation, or shared infra component (DB, cache, queue, host).

DEPENDENCY / TOPOLOGY · D

Known upstream→downstream relation from the service dependency map. Additive, never mandatory.

EVIDENCE SIMILARITY · E

Drain3 log-template match + normalized message similarity between two signals' text.

$$C(A,B) = 0.25 \cdot T + 0.25 \cdot S + 0.15 \cdot C + 0.20 \cdot D + 0.15 \cdot E$$

Weights are explicit starting heuristics, tuned offline against replay fixtures — never claimed as ground truth.

Union-Find — Why It's Used

Once an edge scores $C \geq 0.70$ ("strong edge"), union-find merges its two nodes into the same candidate component in near-linear time — this is what makes bounded candidate generation fast even with hundreds of signals. Union-find **only proposes structure**; it does not decide acceptance. Every resulting component must still pass all four gates below before it becomes an incident.

Four Validation Gates — Candidate → Accepted Incident



BRIDGE CHECK — EXACT RULE (PREVENTS WEAK TRANSITIVE MERGES)

A single "bridge" signal that touches two otherwise-separate sub-clusters is allowed to connect them **only if it has a strong edge ($C \geq 0.70$) to the coherent core of BOTH sides** — not merely to one member of each. If the bridge is strongly tied to only one side, that side keeps it and the other sub-cluster splits off as a separate candidate. This is what stops "signal A relates weakly to B, B relates weakly to C" from chaining A and C into one false incident.

Note on the window: the 5-minute correlation window only bounds which signal pairs are even compared (candidate generation) — it never by itself decides incident membership. Two signals inside the window with weak evidence still fail the gates above; see the worked example on the next page.

Correlation Worked Example, Incident Fingerprint & Continuation

Worked Example — Same 5-Minute Window, Two Outcomes

MERGED → INC-2026-0042

1 DB connection-pool anomaly + 12 application error-log bursts + 1 Grafana latency alert + 3 downstream payment/order timeouts.

Why they merge: all share service=payment→checkout→order, strong topology links (DB→payment→checkout/order), high evidence-similarity on error templates, and land within seconds of each other. All four gates pass.

REJECTED → STAYS SEPARATE

A 17th signal — an unrelated alert on service=recommendation-engine — fires 90 seconds inside the same window.

Why it's rejected: no topology link to payment/checkout/order, near-zero evidence-template similarity, and no strong edge ($C < 0.70$) to the cluster's core. Gate ① never fires for it, so it is never even proposed as candidate structure.

Result: **17 raw signals → 1 accepted incident + 1 correctly-rejected signal** — proving grouping is evidence-based, not merely "everything in the same time bucket."

Incident Fingerprint — Explicit Fields

Environment prod · us-east-1	Service path payment→checkout→order	Components DB connection pool	Topology snapshot DB→payment→checkout/order
Signal templates Drain3 IDs of merged logs	Temporal pattern 10:01:03-10:01:11	Correlation confidence 0.91	Impact severity 85/100 — CRITICAL

Continuation Beyond the Window

New signals arriving after the 5-minute window closes are compared against the **fingerprints of active/recent incidents** — not against raw historical signals. If service-path, component, topology, template, and temporal continuity all score a high similarity match, the new signal **attaches as a continuation** to INC-2026-0042 instead of spawning a duplicate ticket. This is how the design handles incidents that run longer than one correlation window.

Canonical Signal Schema — Compact Example

```
{
  "timestamp": "2026-06-14T10:01:05Z",
  "source": "cloudwatch_logs",
  "environment": "prod / us-east-1",
  "service": "payment-service", "component": "db-connection-pool",
  "signal_type": "error_log_burst", "anomaly_score": 0.86,
  "evidence": "[REDACTED] pool exhausted, template#118"
}
```

Severity, Confidence & Business Criticality

IMPACT SEVERITY (0-100)

0.35·Blast + 0.35·Criticality + 0.20·Trend + 0.10·Magnitude

Blast radius 35%	Scope of affected services
Biz criticality 35%	Checkout/payment > internal admin
Trend 20%	Persisting / worsening direction
Magnitude 10%	Distance from baseline

CORRELATION CONFIDENCE (0-1)

0.35·Density + 0.25·Agreement + 0.20·Topology + 0.20·Temporal

Cluster density 35%	Edge strength within group
Evidence agree. 25%	Independent-source alignment
Topology 20%	Dependency-graph support
Temporal 20%	Timing coherence

WORKED EXAMPLE (CORRECTED)

Blast radius 78 · Business criticality 95 · Trend 88 · Magnitude 70 → **0.35×78 + 0.35×95 + 0.20×88 + 0.10×70 = 27.30 + 33.25 + 17.60 + 7.00 = 85.15 ≈ 85/100 — CRITICAL.** Four aligned sources + strong topology support → **Correlation Confidence 0.91.**

Where Business Criticality Comes From

Criticality scores come from a lightweight, configured **service/path criticality map** — e.g. checkout=95, payment=92, order=88, internal-admin-tool=20. A service that has **no entry** in the map receives a **neutral default score (50)** — never silently treated as critical or trivial — and the incident is flagged for the reviewer to confirm or correct the mapping.

Severity ≠ Confidence — Both Directions

HIGH SEVERITY, LOW CONFIDENCE

A single ambiguous DB alert with a large estimated blast radius but no corroborating logs or Grafana signal — high potential harm, weak grouping evidence. Reviewer sees both numbers, not one blended score.

LOW SEVERITY, HIGH CONFIDENCE

A minor internal-admin latency blip with 6 tightly-correlated signals (same service, template, and topology) — grouping is certain, but business impact is small. No draft needed.

Alert-Fatigue Policy — Decision Matrix

	CONFIDENCE — HIGH	CONFIDENCE — LOW
SEVERITY HIGH	Generate evidence-grounded draft immediately for human review	Generate draft with prominent uncertainty flags — reviewer decides
SEVERITY LOW	Dashboard visibility only — no Jira draft	Dashboard only, lowest priority

Fingerprint match above threshold, at any severity/confidence combination, always attaches as continuation rather than a new draft.

Ticket Drafting — Evidence Provenance & Review Gate

Detection Method, Per Source

METRICS
Statistical baseline (rolling z-score / EWMA) flags deviation beyond a configurable sigma threshold

LOGS / JSON EVENTS
Drain3 template clustering groups messages; a template's frequency spike is the anomaly signal

GRAFANA ALERTS
Passthrough with dedup — Grafana's own rule already fired; we normalize and attach context

EXAMPLE JIRA DRAFT — WITH EVIDENCE PROVENANCE

Title	Payment & Checkout Degradation — Database Connection Pool Saturation
Impact	CRITICAL 85/100 Confidence 0.91
Affected	payment-service · checkout-service · order-service · database
Summary	CloudWatch Connection-pool saturation Logs coincided with error spikes Grafana rising latency Downstream and payment/order timeouts.
Timeline	10:01:03 CloudWatch DB anomaly → :05 Logs error spike → :07 Grafana alert → :09-:11 Downstream timeouts
Investigation	1) pool saturation 2) recent config/deploy changes 3) slow queries/leaks 4) dependent-path recovery

Facts vs. Hypotheses — Visually Separated

✓ OBSERVED EVIDENCE
Connection-pool metric breached baseline · 12 error-log bursts matched template#118 · Grafana latency alert fired · 3 downstream timeouts recorded. Every line here is traceable to a source tag above.

? SUSPECTED — UNVERIFIED
Root cause: database-side connection bottleneck. Marked "suspected, not proven" — the model is prompted to never upgrade this to a stated fact, and the reviewer sees the boundary explicitly.

Human Review Is a Control Plane



CALIBRATION FEEDBACK LOOP
Every reviewer action — **approved / edited / rejected**, with changed fields captured — feeds an offline replay evaluation whose explicit objective is **fewer false merges, fewer missed merges, and fewer unnecessary drafts**. Thresholds and prompts are recalibrated from this signal; production incidents are never retrained mid-event.

Feasibility, Technology Stack & Risks

MVP BOUNDARY — EXPLICIT

The 24-hour prototype prioritizes **replay-based correlation, scoring, and drafting end-to-end**. Live CloudWatch/Grafana/Jira integrations are optional stretch goals — the core proof is the Evidence Graph and gates, not the connectors.

QUANTIFIED DEMO

17 raw signals (1 DB anomaly + 12 log bursts + 1 Grafana alert + 3 downstream timeouts) → **1 accepted incident** → **1 human-reviewed draft**. Negative test: +1 unrelated alert in the same window → correctly rejected.

MVP SUCCESS CRITERIA

- ✔ Correct grouping
- ✔ Rejects unrelated signals
- ✔ Fingerprint continuation
- ✔ Deterministic scoring
- ✔ Evidence traceability
- ✔ Human-only Jira publish

Technology Stack

Ingestion	FastAPI adapters	Detection	Stats + rules + Drain3
Correlation	Graph + union-find	State	SQLite → PostgreSQL
LLM	GPT-4.1-mini	Ticketing	Mock Jira → Jira REST

Top Risks & Mitigations

Over-correlation	Candidate-only components + coherence + bridge checks + false-group replay tests
Under-correlation	Service/component/dependency evidence + configurable window + continuation matching
LLM hallucination	Evidence-only context + fact/hypothesis separation + suspected-status + reviewer gate
Cross-source schema drift	Canonical schema validated at ingestion; malformed/inconsistent fields quarantined, never silently coerced

Standardized Terminology



WHY THIS APPROACH

Time **bounds** candidates; evidence **decides** membership; fingerprints **preserve identity** across a window's lifetime; severity and confidence **separately** quantify harm and uncertainty; and the LLM only **communicates** verified evidence into a human-approved ticket — it never decides what the incident contains.